

Primeros Pasos

El archivo FastAPI más simple podría verse así:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

Copia eso en un archivo `main.py`.

`main.py`

Consejo

FastAPI tiene una extensión oficial para VS Code (y Cursor), que proporciona muchas funcionalidades, incluyendo un explorador de path operations, búsqueda de path operations, navegación CodeLens en tests (saltar a la definición desde los tests), y despliegue y logs de FastAPI Cloud, todo desde tu editor.

Ejecuta el servidor en vivo:

```
$ uv run fastapi dev

FastAPI   Starting development server ■

        Searching for package file structure from directories
        with __init__.py files
        Importing from /home/user/code/awesomeapp

module    ■ main.py

code      Importing the FastAPI app object from the module with
          the following code:

from main import app

app       Using import string: main:app

server    Server started at http://127.0.0.1:8000
server    Documentation at http://127.0.0.1:8000/docs

tip       Running in development mode, for production use:
fastapi run

          Logs:

INFO      Will watch for changes in these directories:
['/home/user/code/awesomeapp']
INFO      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C
          to quit)
INFO      Started reloader process [383138] using WatchFiles
INFO      Started server process [383153]
INFO      Waiting for application startup.
INFO      Application startup complete.
```

En el resultado, hay una línea con algo como:

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Esa línea muestra la URL donde tu aplicación está siendo servida, en tu máquina local.

Revisa

Abre tu navegador en <http://127.0.0.1:8000> .

Verás el response JSON como:

```
{"message": "Hello World"}
```

Documentación interactiva de la API

Ahora ve a <http://127.0.0.1:8000/docs> .

Verás la documentación interactiva automática de la API (proporcionada por Swagger UI):

Documentación alternativa de la API

Y ahora, ve a <http://127.0.0.1:8000/redoc> .

Verás la documentación alternativa automática (proporcionada por ReDoc):

OpenAPI

FastAPI genera un "esquema" con toda tu API utilizando el estándar OpenAPI para definir APIs.

"Esquema"

Un "esquema" es una definición o descripción de algo. No el código que lo implementa, sino solo una descripción abstracta.

"Esquema" de la API

En este caso, OpenAPI es una especificación que dicta cómo definir un esquema de tu API.

Esta definición de esquema incluye los paths de tu API, los posibles parámetros que toman, etc.

"Esquema" de datos

El término "esquema" también podría referirse a la forma de algunos datos, como el contenido JSON.

En ese caso, significaría los atributos del JSON, los tipos de datos que tienen, etc.

OpenAPI y JSON Schema

OpenAPI define un esquema de API para tu API. Y ese esquema incluye definiciones (o "esquemas") de los datos enviados y recibidos por tu API utilizando JSON Schema , el estándar para esquemas de datos JSON.

Revisa el openapi.json

`openapi.json`

Si tienes curiosidad por cómo se ve el esquema OpenAPI en bruto, FastAPI automáticamente genera un JSON (esquema) con las descripciones de toda tu API.

Puedes verlo directamente en: <http://127.0.0.1:8000/openapi.json> .

Mostrará un JSON que empieza con algo como:

```
{
  "openapi": "3.1.0",
  "info": {
    "title": "FastAPI",
    "version": "0.1.0"
  },
  "paths": {
    "/items/": {
      "get": {
        "responses": {
          "200": {
            "description": "Successful Response",
            "content": {
              "application/json": {
                ...

```

Para qué sirve OpenAPI

El esquema OpenAPI es lo que impulsa los dos sistemas de documentación interactiva incluidos.

Y hay docenas de alternativas, todas basadas en OpenAPI. Podrías añadir fácilmente cualquiera de esas alternativas a tu aplicación construida con FastAPI .

También podrías usarlo para generar código automáticamente, para clientes que se comuniquen con tu API. Por ejemplo, aplicaciones frontend, móviles o IoT.

Configura el entrypoint de la app en pyproject.toml

entrypoint

pyproject.toml

Puedes configurar dónde está tu app en un archivo pyproject.toml así:

pyproject.toml

```
[tool.fastapi]
entrypoint = "main:app"
```

Ese entrypoint le dirá al comando fastapi que debe hacer el import de la app así:

entrypoint

fastapi

```
from main import app
```

Si tu código estuviera estructurado así:

```
.
├── backend
│   ├── main.py
│   └── __init__.py
```

Entonces pondrías el entrypoint como:

```
entrypoint
[tool.fastapi]
entrypoint = "backend.main:app"
```

lo cual sería equivalente a:

```
from backend.main import app
```

fastapi dev con path o con la opción de CLI --entrypoint

```
fastapi dev
```

```
--entrypoint
```

También puedes pasar el path del archivo al comando fastapi dev , y adivinará el objeto app de FastAPI que debe usar:

```
fastapi dev
$ uv run fastapi dev main.py
```

O, también puedes pasar la opción --entrypoint al comando fastapi dev :

```
--entrypoint
fastapi dev
$ uv run fastapi dev --entrypoint main:app
```

Pero tendrías que recordar pasar el path\entrypoint correcto cada vez que llames al comando fastapi .

```
fastapi
```

Además, otras herramientas podrían no ser capaces de encontrarlo, por ejemplo la Extensión de VS Code o FastAPI Cloud , así que se recomienda usar el entrypoint en pyproject.toml .

```
entrypoint
```

```
pyproject.toml
```

Despliega tu app (opcional)

Opcionalmente puedes desplegar tu app de FastAPI en FastAPI Cloud con un solo comando. ■

```
$ uv run fastapi deploy

Deploying to FastAPI Cloud...

■ Deployment successful!

■ Ready the chicken! Your app is ready at https://myapp.fastapicloud.dev
```

La CLI detectará automáticamente tu aplicación de FastAPI y la desplegará en la nube. Si no has iniciado sesión, se abrirá tu navegador para completar el proceso de autenticación.

¡Eso es todo! Ahora puedes acceder a tu app en esa URL. ■

Recapitulación, paso a paso

Paso 1: importa FastAPI

FastAPI

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

FastAPI es una clase de Python que proporciona toda la funcionalidad para tu API.

FastAPI

Detalles técnicos

FastAPI es una clase que hereda directamente de Starlette .

FastAPI

Starlette

Puedes usar toda la funcionalidad de Starlette con FastAPI también.

FastAPI

Paso 2: crea una "instance" de FastAPI

FastAPI

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

Aquí la variable app será una "instance" de la clase FastAPI .

app

FastAPI

Este será el punto principal de interacción para crear toda tu API.

Paso 3: crea una path operation

Path

"Path" aquí se refiere a la última parte de la URL empezando desde la primera / .

/

Así que, en una URL como:

`https://example.com/items/foo`

...el path sería:

`/items/foo`

Nota

Un "path" también es comúnmente llamado "endpoint" o "ruta".

Mientras construyes una API, el "path" es la forma principal de separar "concerns" y "resources".

Operación

"Operación" aquí se refiere a uno de los "métodos" HTTP.

Uno de:

- POST

POST

- GET

GET

- PUT

PUT

- DELETE

DELETE

...y los más exóticos:

- OPTIONS

OPTIONS

- HEAD

HEAD

- PATCH

PATCH

- TRACE

TRACE

En el protocolo HTTP, puedes comunicarte con cada path usando uno (o más) de estos "métodos".

Al construir APIs, normalmente usas estos métodos HTTP específicos para realizar una acción específica.

Normalmente usas:

- POST : para crear datos.

POST

- GET : para leer datos.

GET

- PUT : para actualizar datos.

PUT

- DELETE : para eliminar datos.

DELETE

Así que, en OpenAPI, cada uno de los métodos HTTP se llama una "operación".

Vamos a llamarlas "operaciones" también.

Define un path operation decorator

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

El `@app.get("/")` le dice a FastAPI que la función justo debajo se encarga de manejar requests que vayan a:

`@app.get("/")`

- el path /

/

- usando una get operación

get

Información sobre `@decorator`

`@decorator`

Esa sintaxis `@algo` en Python se llama un "decorador".

`@algo`

Lo pones encima de una función. Como un bonito sombrero decorativo (supongo que de ahí viene el término).

Un "decorador" toma la función de abajo y hace algo con ella.

En nuestro caso, este decorador le dice a FastAPI que la función de abajo corresponde al path / con una operation get .

/

get

Es el " path operation decorator ".

También puedes usar las otras operaciones:

- @app.post()

@app.post()

- @app.put()

@app.put()

- @app.delete()

@app.delete()

Y los más exóticos:

- @app.options()

@app.options()

- @app.head()

@app.head()

- @app.patch()

@app.patch()

- @app.trace()

@app.trace()

Consejo

Eres libre de usar cada operación (método HTTP) como quieras.

FastAPI no fuerza ningún significado específico.

La información aquí se presenta como una guía, no un requisito.

Por ejemplo, cuando usas GraphQL normalmente realizas todas las acciones usando solo operaciones POST .

POST

Paso 4: define la path operation function

Esta es nuestra " path operation function ":

- path : es / .

/

- operation : es get .

get

- function : es la función debajo del "decorador" (debajo de @app.get("/")).

```
@app.get("/")  
    from fastapi import FastAPI  
  
    app = FastAPI()  
  
    @app.get("/")  
    async def root():  
        return {"message": "Hello World"}
```

Esta es una función de Python.

Será llamada por FastAPI cuando reciba un request en la URL "/" usando una operación GET .

/

GET

En este caso, es una función async .

async

También podrías definirla como una función normal en lugar de async def :

```
async def  
    from fastapi import FastAPI  
  
    app = FastAPI()  
  
    @app.get("/")  
    def root():  
        return {"message": "Hello World"}
```

Nota

Si no sabes la diferencia, Revisa la sección Async: "¿Tienes prisa?" .

Paso 5: retorna el contenido

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/")  
async def root():  
    return {"message": "Hello World"}
```

Puedes retornar un dict , list , valores singulares como str , int , etc.

dict

list

str

int

También puedes retornar modelos de Pydantic (verás más sobre eso más adelante).

Hay muchos otros objetos y modelos que serán automáticamente convertidos a JSON (incluyendo ORMs, etc). Intenta usar tus favoritos, es altamente probable que ya sean compatibles.

Paso 6: Despliegalo

Despliega tu app en FastAPI Cloud con un solo comando: `fastapi deploy` . ■

`fastapi deploy`

Sobre FastAPI Cloud

FastAPI Cloud está construido por el mismo autor y equipo detrás de FastAPI .

Agiliza el proceso de construir , desplegar y acceder a una API con el mínimo esfuerzo.

Trae la misma experiencia de desarrollador de construir apps con FastAPI a desplegarlas en la nube. ■

FastAPI Cloud es el sponsor principal y proveedor de financiación para los proyectos open source de FastAPI and friends . ■

Despliega en otros proveedores cloud

FastAPI es open source y basado en estándares. Puedes desplegar apps de FastAPI en cualquier proveedor cloud que elijas.

Sigue las guías de tu proveedor cloud para desplegar apps de FastAPI con ellos. ■

Recapitulación

- Importa FastAPI .

`FastAPI`

- Crea una instance app .

`app`

- Escribe un path operation decorator usando decoradores como `@app.get("/")` .

`@app.get("/")`

- Define una path operation function ; por ejemplo, `def root(): ...` .

`def root(): ...`

- Ejecuta el servidor de desarrollo usando el comando `fastapi dev` .

`fastapi dev`

- Opcionalmente, despliega tu app con `fastapi deploy` .

fastapi deploy